SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 20.2

Program	**: B. Tech (CSE)**
Specialization	**: --**
Course Title	**: AI Assisted coding**
Course Code	**:**
Semester	**II**
Academic Session	**: 2025-2026**
Name of Student	**: Naniprasad**
Enrollment No.	**: 2403A51L11**
Batch No.	**: 51**
Date	**:10-04-2026**

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Prompt:Generate a simple Python program for user registration and login that stores usernames and passwords.

Step 1:

First, create a basic version where passwords are stored in plain text in a file or dictionary.

Step 2:

Analyze the program and identify security issues with storing passwords in plain text.

Step 3:

Refactor the program to improve security by:

- Using password hashing (hashlib or bcrypt)
- Storing only hashed passwords instead of plain text
- Ensuring secure comparison during login

Step 4:

Update the login system to validate user credentials using hashed passwords.

Ensure:

- Proper input handling
- Clear code structure
- Comments explaining security improvements
- The program works correctly for both registration and login

Provide:

1. Original insecure code
2. Explanation of security risks
3. Secure improved code
4. Sample test cases and output

CODE:

```python
import os
import hashlib
# Insecure version: Storing passwords in plain text
def register_user_insecure(username, password):
    with open('users_insecure.txt', 'a') as file:
        file.write(f"{username}:{password}\n")

def login_user_insecure(username, password):
    with open('users_insecure.txt', 'r') as file:
        for line in file:
            stored_username, stored_password = line.strip().split(':')
            if stored_username == username and stored_password == password:
                return "Login successful!"
    return "Login failed!"
# Secure Improved Code
def hash_password(password):
    # Hash the password using SHA-256
    return hashlib.sha256(password.encode()).hexdigest()
def register_user_secure(username, password):
    hashed_password = hash_password(password)
    with open('users_secure.txt', 'a') as file:
        file.write(f"{username}:{hashed_password}\n")
def login_user_secure(username, password):
    hashed_password = hash_password(password)
    with open('users_secure.txt', 'r') as file:
        for line in file:
            stored_username, stored_hashed_password = line.strip().split(':')
            if stored_username == username and stored_hashed_password == hashed_password:
                return "Login successful!"
    return "Login failed!"
# Sample Test Cases and Output
# Test Case 1: Registering and logging in with the secure version
register_user_secure("user1", "password123")
print(login_user_secure("user1", "password123"))  # Output: Login successful!
print(login_user_secure("user1", "wrongpassword"))  # Output: Login failed!
# Test Case 2: Attempting to log in with a non-existent user
print(login_user_secure("user2", "password123"))  # Output: Login failed!
```

Output:

```
/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
Login successful!
Login failed!
Login failed!
akash@AKASHs-MacBook-Air ai_assis %  source /Users/akash/Desktop/ai_assis/.venv/bin/activate
```

Explanation:# 1. Storing passwords in plain text is highly insecure because if the file is accessed by unauthorized users, they can easily read all usernames and passwords.
2. If the file is compromised, it can lead to identity theft and unauthorized access to user accounts.
3. Plain text passwords can be easily reused across different platforms, increasing the risk of widespread account breaches.

Task 2 – Secure API Key Management:

Prompt:

Generate a Python script that connects to an external API (for example, a weather API or any public API) using an API key.

Step 1:

First, create a basic version where the API key is hardcoded directly in the source code.

Step 2:

Analyze the program and identify the security risks of hardcoding API keys in the code.

Step 3:

Refactor the program to improve security by:

- Storing the API key in environment variables instead of hardcoding it
- Accessing the API key securely using Python (e.g., using os.getenv())
- Ensuring the API key is not exposed in the source code

Step 4:

Update the script to successfully fetch data from the API using the securely stored API key.

Ensure:

- Proper error handling (e.g., missing API key, invalid key)
- Clear and readable code
- Comments explaining the security improvements
- The program works correctly with environment variables

Provide the following:

1. Original insecure Python code with hardcoded API key
2. Explanation of security risks
3. Secure improved code using environment variables
4. Sample execution/output showing successful API response

CODE:

```python
import os
import requests
# Insecure version: Hardcoded API key
def fetch_weather_insecure(city):
    api_key = "your_hardcoded_api_key"
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    else:
        return "Failed to fetch weather data."
# Secure Improved Code
def fetch_weather_secure(city):
    api_key = os.getenv("OPENWEATHER_API_KEY")
    if not api_key:
        return "API key is missing. Please set the OPENWEATHER_API_KEY environment variable."
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    else:
        return "Failed to fetch weather data."
# Sample Execution/Output
# To test the secure version, ensure you have set the environment variable OPENWEATHER_API_KEY
# For example, in a terminal you can set it like this:
# export OPENWEATHER_API_KEY=your_actual_api_key
print(fetch_weather_secure("London"))  # Output: Weather data for London or error message if API key is missing/invalid
```

Output:

```
/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
API key is missing. Please set the OPENWEATHER_API_KEY environment variable.
```

Explanation of Security Risks

1. Hardcoding API keys in the source code is a significant security risk because if the code is shared or uploaded to a public repository, the API key can be easily accessed by unauthorized users.

2. If the API key is compromised, it can lead to unauthorized access to the API, potential abuse of the service, and unexpected charges if the API has usage limits.

3. Storing API keys in environment variables helps to keep them secure and separate from the source code, reducing the risk of accidental exposure.

Task 3 – Weak Encryption Detection

Prompt:Generate a simple Python program that performs encryption and decryption using a weak or outdated technique (for example, Caesar cipher or base64 encoding).

Step 1:

Create the initial program using a basic/reversible encryption method.

Step 2:

Analyze the program and identify the weaknesses of the encryption approach, such as:
- Easy to reverse without a key
- Lack of strong security
- Not suitable for protecting sensitive data

Step 3:
Refactor the program to improve security by:
- Using a stronger encryption method (such as Fernet from the cryptography library)
- Implementing proper key generation and management
- Ensuring that data cannot be decrypted without the correct key

Step 4:
Update the program to securely encrypt and decrypt data using the improved method.

Ensure:
- Clear and readable code
- Proper use of Python libraries for encryption
- Comments explaining security improvements
- The program works correctly for both encryption and decryption

Provide the following:
1. Original insecure encryption code
2. Explanation of weaknesses
3. Improved secure encryption code
4. Sample input and output demonstrating encryption and decryption

CODE:

```python
def caesar_cipher_encrypt(plain_text, shift):
    encrypted_text = ""
    for char in plain_text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                encrypted_char = chr((ord(char) - ord('a') + shift_amount) % 26 + ord('a'))
            else:
                encrypted_char = chr((ord(char) - ord('A') + shift_amount) % 26 + ord('A'))
            encrypted_text += encrypted_char
        else:
            encrypted_text += char
    return encrypted_text
def caesar_cipher_decrypt(encrypted_text, shift):
    decrypted_text = ""
    for char in encrypted_text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                decrypted_char = chr((ord(char) - ord('a') - shift_amount) % 26 + ord('a'))
            else:
                decrypted_char = chr((ord(char) - ord('A') - shift_amount) % 26 + ord('A'))
            decrypted_text += decrypted_char
        else:
            decrypted_text += char
    return decrypted_text

from cryptography.fernet import Fernet
# Generate a key for encryption and decryption
def generate_key():
    return Fernet.generate_key()
# Encrypt the data using the generated key
def encrypt_data(plain_text, key):
    fernet = Fernet(key)
    encrypted_text = fernet.encrypt(plain_text.encode())
    return encrypted_text
# Decrypt the data using the same key
def decrypt_data(encrypted_text, key):
```

```python
# Encrypt the data using the generated key
def encrypt_data(plain_text, key):
    fernet = Fernet(key)
    encrypted_text = fernet.encrypt(plain_text.encode())
    return encrypted_text
# Decrypt the data using the same key
def decrypt_data(encrypted_text, key):
    fernet = Fernet(key)
    decrypted_text = fernet.decrypt(encrypted_text).decode()
    return decrypted_text
# Sample Input and Output
key = generate_key()
print(f"Generated Key: {key}")
original_text = "Hello, World!"
print(f"Original Text: {original_text}")
encrypted_text = encrypt_data(original_text, key)
print(f"Encrypted Text: {encrypted_text}")
decrypted_text = decrypt_data(encrypted_text, key)
print(f"Decrypted Text: {decrypted_text}")
```

```
Generated Key: b'fVBSuqY2USwUPoW0bfqmwMQJGkH5CJIO41YOIsh1UUw='
Original Text: Hello, World!
Encrypted Text: b'gAAAAABp2INJ-FLo-4JLVqj0HeXgTmDg7afc8Ymxco2io532jFdOLRZ7vZD4pi6w0hmAvanbiUXP1nyyHCtXUXQL4oaq3Dn4Hw=='
Decrypted Text: Hello, World!
akash@AKASHs-MacBook-Air ai_assis %  source /Users/akash/Desktop/ai_assis/.venv/bin/activate
```

```
# Explanation of Weaknesses
# 1. The Caesar cipher is a very weak encryption method because it can be easily
# decrypted without a key by trying all possible shifts (brute-force attack).
# 2. It does not provide any real security for sensitive data, as it can be easily broken with simple frequency analysis or pattern recognition.
# 3. It is not suitable for protecting sensitive information, as it can be easily reversed by anyone who has access to the encrypted text.
# Improved Secure Encryption Code
```

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service:
Prompt:Generate a simple web service in Python (for example, a REST API using Flask) that performs basic operations such as user login, data submission, or file upload.

Step 1:
Create an initial version of the web service that may contain common security vulnerabilities (e.g., no input validation, hardcoded credentials, no authentication, or insecure file handling).

Step 2:
Perform a structured security review of the generated code and identify at least three vulnerabilities, such as:
- SQL Injection
- Cross-Site Scripting (XSS)
- Hardcoded credentials
- Lack of input validation
- Insecure file upload handling

Step 3:
Refactor the program to improve security by:
- Validating and sanitizing user inputs
- Using secure authentication mechanisms
- Avoiding hardcoded credentials
- Implementing proper error handling
- Applying secure coding best practices

Step 4:

Update the web service to fix the identified vulnerabilities and ensure secure functionality.

Ensure:

- Clean and readable code
- Proper use of libraries (e.g., Flask security practices)
- Comments explaining each security improvement
- The application works correctly after applying fixes

Provide the following:

1. Original insecure web service code
2. List and explanation of at least three vulnerabilities
3. Secure improved version of the code
4. Comparison between insecure and secure implementations
5. Sample test cases or outputs demonstrating improved security

CODE:

```python
# Original Insecure Web Service Code
from flask import Flask, request, jsonify
import os
from werkzeug.utils import secure_filename
from werkzeug.security import check_password_hash, generate_password_hash

app = Flask(__name__)

# Create upload folder
UPLOAD_FOLDER = 'uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# Store credentials securely
ADMIN_USERNAME = os.getenv('ADMIN_USERNAME', 'admin')
ADMIN_PASSWORD_HASH = os.getenv('ADMIN_PASSWORD_HASH', generate_password_hash('password123'))

@app.route('/login', methods=['POST'])
def login():
    username = request.form.get('username')
    password = request.form.get('password')

    # Input validation
    if not username or not password:
        return jsonify({'message': 'Missing username or password'}), 400

    if username == ADMIN_USERNAME and check_password_hash(ADMIN_PASSWORD_HASH, password):
        return jsonify({'message': 'Login successful'})
    else:
        return jsonify({'message': 'Invalid credentials'}), 401

# File upload validation
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/upload', methods=['POST'])
def upload_file():
```

```python
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return jsonify({'message': 'No file part'}), 400

    file = request.files['file']

    if file.filename == '':
        return jsonify({'message': 'No selected file'}), 400

    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        file.save(os.path.join(UPLOAD_FOLDER, filename))
        return jsonify({'message': 'File uploaded successfully'})
    else:
        return jsonify({'message': 'Invalid file type'}), 400

if __name__ == '__main__':
    app.run(debug=True, use_reloader=False)
```

```
/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
 * Serving Flask app 'lab_20.2'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
^C%
akash@AKASHs-MacBook-Air ai_assis %  source /Users/akash/Desktop/ai_assis/.venv/bin/activate
(.venv) akash@AKASHs-MacBook-Air ai_assis %
```

Task 5 – Exception Handling Improvement:

Prompt:Generate a simple Python program that takes user input and performs basic arithmetic operations such as addition, subtraction, multiplication, and division.

Step 1:

Create an initial version of the program without exception handling, where user input is directly used.

Step 2:

Analyze the program and identify possible runtime errors, such as:

- Invalid input (e.g., entering text instead of numbers)
- Division by zero
- Missing or incorrect inputs

Step 3:

Refactor the program to improve reliability by:

- Using try-except blocks to handle exceptions
- Catching specific errors such as ValueError and ZeroDivisionError
- Providing meaningful and user-friendly error messages

Step 4:

Update the program so that it continues to run smoothly without crashing when errors occur.

Ensure:

- Clean and readable code
- Proper use of exception handling
- Clear comments explaining error handling improvements
- The program works correctly for both valid and invalid inputs

Provide the following:

1. Original insecure program (without exception handling)
2. List of possible runtime errors
3. Improved program with try-except blocks
4. Sample test cases and outputs for both valid and invalid inputs

CODE:

```python
def perform_operation(num1, num2, operation):
    if operation == 'add':
        return num1 + num2
    elif operation == 'subtract':
        return num1 - num2
    elif operation == 'multiply':
        return num1 * num2
    elif operation == 'divide':
        return num1 / num2
    else:
        return None

def main():
    try:
        num1 = float(input("Enter the first number: "))
        num2 = float(input("Enter the second number: "))
        operation = input("Enter operation (add, subtract, multiply, divide): ").lower()

        result = perform_operation(num1, num2, operation)

        if result is None:
            print("Error: Invalid operation entered.")
        else:
            print(f"Result: {result}")

    except ValueError:
        print("Error: Please enter valid numeric values.")

    except ZeroDivisionError:
        print("Error: Division by zero is not allowed.")

    except KeyboardInterrupt:
        print("\nProgram interrupted by user.")

    except Exception as e:
        print("Unexpected error:", e)

if __name__ == "__main__":
```

Ouput:

```
/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_20.2.py
Enter the first number: 34
Enter the second number: 56
Enter operation (add, subtract, multiply, divide): ^C
Program interrupted by user.
akash@AKASHs-MacBook-Air ai_assis %  source /Users/akash/Desktop/ai_assis/.venv/bin/activate
(.venv) akash@AKASHs-MacBook-Air ai_assis %
```

```
# Expalnation of error handling improvements:
# 1. The program now includes try-except blocks to catch specific exceptions such as ValueError for invalid numeric input and ZeroDivisionError for division by zero.
# 2. User-friendly error messages are provided to guide the user in case of invalid input or operations.
# 3. The program continues to run smoothly without crashing when errors occur, allowing the user to correct their input and try again.
```